DOCUMENTACION TECNICA · TOTAL CLEAN CAR

Manual de administrador — Total Clean Car

Proyecto: Total Clean Car

Version: 2.1

Fecha: 11/10/2026

Elaborado por: Auditoria tecnica

Versión 2.1 · 2026-10-08 · Reemplaza a la 2.0 (2026-09-27) y a la 1.0 (2026-09-15, "LavadoCarros")

Novedades de la 2.1: vehículos de cada cliente visibles desde Clientes (sección 2.1), Dashboard y Reportes ahora distintos y con filtro por mes (sección 9), sesión con renovación automática (sección 10.1), despliegue con `scripts/desplegar.sh` (sección 11) y **respaldo automático diario** (sección 13).

Manual técnico: roles, catálogo, reglas de agenda, parámetros, despliegue y mantenimiento. El uso de las pantallas está en `07_MANUAL_USUARIO/Manual_Usuario.md` (secciones 6 y 7). Los endpoints completos, en Swagger: `http://127.0.0.1:3042/api/docs`.

1. Usuarios y roles

Cuatro roles: `admin`, `operador`, `lavador`, `cliente`. Distribución al 2026-09-27: 7 clientes, 3 lavadores, 1 operador, 1 admin.

Rol	Alcance	Inicio al entrar
`admin`	administración completa	`/dashboard` (en el celular o la APK, `/agenda`)
`operador`	agenda, reservas, asignaciones, validación de pagos, catálogo	`/dashboard` (en el celular o la APK, `/agenda`)
`lavador`	sus trabajos y carga de evidencia	`/mis-trabajos`
`cliente`	sus reservas, vehículos, pagos y calificaciones	`/mis-reservas`

Suspender una cuenta corta el acceso de inmediato. El sistema revalida el estado en cada petición, así que no hay que esperar a que caduque el token. Basta poner `estado` distinto de `activo` en `Usuario`, o en la ficha de `Cliente` o `Lavador`.

Advertencia: el rol es texto libre en la base. Al crear usuarios **por fuera de la aplicación**, un valor mal escrito (`"Admin"` en vez de `"admin"`) deja la cuenta sin permisos, sin mensaje de error. Usar siempre minúsculas y exactamente uno de los cuatro valores.

1.1 Credenciales

Estas tres acciones son **solo del `admin`**:

Acción	Endpoint
Crear el acceso o restablecer la contraseña de un cliente	`POST /api/clientes/:id/acceso`
Crear o editar la cuenta de acceso de un lavador	`PUT /api/lavadores/:id/usuario`
Restablecer la contraseña de un lavador	`POST /api/lavadores/:id/usuario/reset`

El sistema genera una contraseña temporal que se muestra **una sola vez**.

> **Pendiente de seguridad (2026-09-27):** `admin`, `operador`, `lavador1` y `lavador2` siguen con las contraseñas iniciales de `backend/prisma/seed.js`, que está publicado en GitHub. Cambiarlas antes de dar acceso a más personas o de publicar la app con dominio. Los clientes `*@demo.com` son de demostración.

2. Catálogos

Catálogo	Entidad	Registros al 2026-09-27
Servicios	`TipoServicio`	6
Tipos de vehículo	`TipoVehiculo`	4 (moto, liviano, SUV, camioneta)
Precios por tipo	`PrecioServicio`	18 activos
Servicios adicionales	`ServicioAdicional`	0
Estacionamientos	`Estacionamiento`	5
Plazas y bahías	`Plaza`	42
Planes	`Plan`	1 (2 suscripciones activas)

Ningún registro de catálogo se borra: se desactiva (`activo = false` o `estado = 'inactivo'`). Las reservas guardan su propia copia de lo que se cobró. El catálogo lo pueden editar `admin` y `operador`.

2.1 Vehículos de cada cliente

En **Clientes**, la columna *Vehículos* es un botón: el número abre la lista del cliente debajo de su fila. De cada vehículo se ve la placa, el tipo, la marca y modelo, el color y si está de baja.

Desde ahí mismo, sin cambiar de pantalla:

Acción	Para qué sirve
Agregar vehículo	el cliente que llega sin tener el carro cargado; evita tener que pedirle que lo registre él
Lápiz (cambiar el tipo)	corregir un tipo mal puesto o **completar el que falta**
Prohibido / flecha	dar de baja o reactivar. No se borra: las reservas anteriores lo siguen mostrando

> **Un vehículo "sin tipo" aparece marcado en ámbar y con un aviso.** No es un detalle estético: sin tipo no hay precio (sección 3) y **ese cliente no puede reservar con ese vehículo**. Son los que se cargaron antes de que existiera el catálogo de tipos, en septiembre de 2026. Es la primera causa de la llamada "*no me deja reservar*": abrir su fila y ponerle el tipo lo resuelve en el acto.

Los vehículos vienen en la misma respuesta que la lista de clientes, así que abrir la fila no consulta nada: es instantáneo.

3. Precios por tipo de vehículo

El precio y la duración **no** están en `TipoServicio`: están en `PrecioServicio`, una fila por servicio y tipo de vehículo (única por `id_tipo_servicio` + `id_tipo_vehiculo`).

- Un servicio se ofrece a un tipo solo si existe esa fila, con `activo = true`, y el tipo está activo.
- La duración de la fila define cuánto se bloquea en la agenda.
- `GET /api/public/servicios` devuelve cada servicio con sus precios y `precio_desde`; omite los que no tienen ningún precio.
- `POST` y `PUT /api/servicios` reciben `precios: [{ id_tipo_vehiculo, precio, duracion_min }]`. Un tipo que deja de venir queda desactivado, no borrado. No se acepta un servicio sin ningún precio.
- Tipos de vehículo: `GET/POST/PUT /api/tipos-vehiculo` (interno) y `GET /api/public/tipos-vehiculo`. El `codigo` se genera a partir del nombre.

`Vehiculo.id_tipo_vehiculo` es obligatorio al registrar (`POST /api/vehiculos`). Admite `null` solo por los vehículos anteriores al catálogo: **3 vehículos** siguen sin tipo y no pueden reservar hasta que su dueño o el operador lo complete. El cliente puede editar sus propios vehículos (`PUT /api/vehiculos/:id`), pero no su estado.

> **Pendiente (2026-09-27):** el tipo **moto** no tiene ningún precio, así que ningún servicio se le ofrece. La migración no inventó tarifas: se cargan desde **Servicios → Servicios y precios**.

4. Servicios adicionales

`ServicioAdicional` (nombre, precio, `duracion_min` extra). CRUD interno en `/api/adicionales`; lectura pública en `/api/public/adicionales`.

Al reservar, cada adicional elegido se copia a `ReservaAdicional` (nombre, precio y minutos): cambiar el catálogo después no altera reservas ya hechas.

`Reserva.precio_final` = precio del servicio para el tipo de vehículo (0 si lo cubre un plan) + adicionales. **Los planes no cubren adicionales.**

5. Cotización y creación de reservas

El precio y la duración los calcula siempre el backend (`services/cotizacion.service.js`); lo que muestra la pantalla sale de la misma cuenta.

- `POST /api/reservas/cotizar` — vista previa: `{ id_vehiculo, id_tipo_servicio, adicionales }`.
- `POST /api/reservas` — crea la reserva, con `adicionales: [ids]` opcional.

Validaciones al crear, en este orden: cliente activo, vehículo del cliente y activo, servicio ofrecido para el tipo de vehículo, estacionamiento (en expreso), fecha no pasada, **hora no empezada** (si es hoy), dentro del horario de atención del sitio, y cupo disponible. El cupo se verifica en una transacción **serializable**: si dos clientes piden el último cupo a la vez, uno lo obtiene y el otro recibe `409`.

6. Agenda y capacidad

Modalidad	Capacidad por franja
Expreso	`Estacionamiento.capacidad_expreso` del sitio; si es 0, la cantidad de lavadores activos
Profunda	bahías (`Plaza.tipo = 'bahia_lavado'`) disponibles

- **Horarios ofrecidos:** empiezan en la apertura del sitio y avanzan cada `duracion_franja_min` (60 por defecto) para servicios que duran eso o más; los más cortos, cada 30 minutos. Así los horarios caen en :00 y :30.
- **Franjas pasadas:** hoy, las que ya empezaron salen con `pasada: true`, sin cupos, y no se pueden reservar.
- **La disponibilidad y la creación usan la misma regla** (`services/agenda.service.js`). Hasta el 2026-09-27, crear contaba la capacidad global y un sitio con capacidad propia podía aceptar reservas encimadas.
- **Bloqueos manuales:** `POST /api/agenda/bloqueos` y `DELETE /api/agenda/bloqueos/:id`. Bloquean ambas modalidades en todos los sitios.
- **Asignar lavador:** `PUT /api/reservas/:id/asignar-lavador`. Rechaza con `409` si el lavador ya tiene otro trabajo solapado ese día.
- **Estados:** `solicitada → confirmada → en_proceso → completada`, con salidas a `cancelada` (desde solicitada o confirmada) y `no_asistio` (desde confirmada o en proceso). `PUT /api/reservas/:id/no-asistio` existe, pero **el panel no tiene botón** para marcarlo.

7. Fechas y zona horaria

`Reserva.fecha` es un día, guardado como **medianoche UTC** (`2026-09-27T00:00:00Z`). El contenedor corre en UTC; "hoy", "ahora", los KPIs del día y los recordatorios se calculan en `APP_TZ` (por defecto `America/Guayaquil`). Todo pasa por `backend/src/utils/fechas.js` y `frontend/src/utils/fechas.ts`.

> Antes del 2026-09-27 el navegador (UTC−5) mostraba cada reserva un día antes, y desde las 19:00 el servidor ya contaba el día siguiente. No volver a usar `new Date(fecha)` con `setHours`, ni `toLocaleDateString()` sobre `Reserva.fecha`.

8. Validación de pagos

Los comprobantes que sube el cliente quedan `en_verificacion` y no cuentan como ingreso. Se listan con `GET /api/pagos/pendientes` y se resuelven con `PUT /api/reservas/:id/pagos/:pagoId/verificar`. Un comprobante es un archivo de hasta 5 MB (JPG, PNG, WEBP, GIF o PDF).

> La **pasarela de tarjeta no está conectada**: `POST /api/reservas/:id/pagos/tarjeta` deja el pago pendiente, y la pantalla del cliente no ofrece tarjeta. Antes de conectarla, resolver SEC-02: el webhook no valida firma.

9. Dashboard y Reportes

Hasta el 2026-10-08 eran casi la misma pantalla: el Dashboard mostraba las tarjetas de Reportes sin las gráficas, o sea un subconjunto. Ahora cada una responde una pregunta distinta.

	Dashboard	**Reportes**
Pregunta	¿cómo vamos **ahora**?	¿cómo nos fue **en este mes**?
Período	fijo: hoy / esta semana / este mes, y los últimos 30 días en las gráficas	el mes que se elija
Se configura	nada: se abre y se lee	selector de mes
Exporta	no	Excel y PDF **del mes que está en pantalla**
Para quién	el día a día del operador	revisar, comparar y cerrar el mes

9.1 Dashboard

Cinco tarjetas (lavados de hoy, del mes, cobrado del mes, ocupación y calificación) y dos gráficas de los **últimos 30 días**: ingresos por día y lavados por servicio. Las dos miran la misma ventana a propósito, para que se lean juntas.

Si hay solicitudes sin confirmar aparece un aviso ámbar que **lleva a Reservas ya filtrado**: un aviso que no se puede atender es ruido.

9.2 Reportes

El selector de mes está arriba y manda sobre todo lo de abajo. Las flechas se mueven de a un mes; el campo de mes salta a uno lejano. No se puede elegir un mes futuro.

Facturado y cobrado no son lo mismo, y es la confusión más común al cerrar el mes:

	Qué suma
Facturado	el precio de los lavados **hechos** en el mes
Cobrado	los **pagos aprobados** con fecha dentro del mes

Un lavado del 28 de septiembre cobrado el 2 de octubre factura en septiembre y cobra en octubre. Cuando los dos números difieren, la pantalla lo explica sola.

El **ticket promedio** es facturado ÷ lavados. Si no hubo lavados muestra "—", no "$0.00": cero haría pensar que se trabajó gratis.

Las gráficas de barras acortan los nombres largos con puntos suspensivos; el nombre completo y el número exacto están en la tabla debajo de cada una. La gráfica de ingresos tiene un "Ver el detalle por día" plegado, con la misma información en texto.

9.3 Endpoints

Reporte	Endpoint	Acepta `desde` / `hasta`
KPIs en vivo (Dashboard)	`GET /api/reportes/kpis`	no
Resumen del período (Reportes)	`GET /api/reportes/resumen`	**sí**
Ingresos por día	`GET /api/reportes/ingresos`	**sí**
Por estacionamiento	`GET /api/reportes/por-estacionamiento`	**sí**
Por servicio	`GET /api/reportes/por-servicio`	**sí**
Exportar Excel	`GET /api/reportes/exportar/excel`	**sí**
Exportar PDF	`GET /api/reportes/exportar/pdf`	**sí**
Acta de servicio	`GET /api/reportes/reserva/:id/acta.pdf`	—

Las fechas van como `YYYY-MM-DD` e **incluyen los dos extremos**. Sin ellas, los agregados devuelven el histórico completo.

El acta incluye el tipo de vehículo y los adicionales con su precio. Los PDF llevan el logo desde `backend/assets/logo-mark.png`.

10. Parámetros del sistema

Todo en `.env`, junto a `docker-compose.yml`. **Es de `server-dt` con permisos `600`**: no volver a dejarlo legible por todos (hasta el 2026-09-27 era `644` de root). Un cambio requiere recrear el contenedor: `docker compose up -d backend`.

Variable	Efecto
`JWT_EXPIRES_IN`	duración del token de acceso (hoy `15m`) — ver 10.1
`REFRESH_EXPIRA_DIAS`	días que dura la sesión sin volver a escribir la contraseña (hoy `7`)
`APP_TZ`	zona horaria del negocio (por defecto `America/Guayaquil`)
`CORS_ORIGINS`	orígenes permitidos, separados por coma; vacío = abierto. La web no lo necesita; la APK usa el origen `http://localhost`
`RATE_LIMIT_GENERAL`	límite de peticiones general
`RATE_LIMIT_OFF`	desactiva los límites
`SMTP_*`	envío de correo; el remitente se muestra como "Total Clean Car"
`TWILIO_*`	WhatsApp y SMS

> `RATE_LIMIT_OFF` **desactiva la protección contra fuerza bruta.** Usarlo solo en pruebas y nunca dejarlo activo en producción.

Los recordatorios salen todos los días a las **18:00 hora de** `APP_TZ`, para las reservas del día siguiente.

10.1 Cómo funciona la sesión (desde el 2026-10-08)

Antes el token duraba 24 horas y no se podía anular: quien se quedaba con el de un operador tenía un día entero de acceso, y "cerrar sesión" solo lo borraba de ese navegador. Ahora son dos piezas:

Pieza	Dura	Qué pasa si la roban
Token de **acceso**	`JWT_EXPIRES_IN` = 15 min	sirve 15 minutos y caduca solo
Token de **refresco**	`REFRESH_EXPIRA_DIAS` = 7 días	se detecta al reusarlo y corta la sesión

El usuario no nota nada: el navegador y la APK renuevan el acceso por detrás. Sigue entrando una vez por semana, igual que antes.

Lo que sí cambia para el administrador:

- **Cerrar sesión ahora cierra de verdad**, también en el servidor.
- **Restablecer la contraseña de un cliente o un lavador** (botón de la llave en su ficha) **lo echa de todos lados en el acto**. Antes podía seguir trabajando con el token viejo hasta un día entero.
- Las sesiones vivas están en la tabla `RefreshToken`. Para cortar todas las de una persona a mano:

  ```
  docker exec lavado_de_carros-postgres-1 psql -U lavado_user -d lavado_db \
  -c "UPDATE \"RefreshToken\" SET revocado = true WHERE id_usuario = ;"
  ```

- **Cambiar `JWT_SECRET` cierra todas las sesiones de todos.** Es lo esperado al rotarlo, pero conviene hacerlo fuera de horario de atención.
- Al desplegar esto por primera vez, quien tenía una sesión abierta con el esquema anterior entra una vez más y listo. No hay que avisar a nadie.

11. Despliegue y migraciones

El esquema **no** usa `prisma migrate` (no hay tabla `_prisma_migrations`): se aplicó con `db push`. Los cambios de esquema son SQL a mano en `backend/prisma/migraciones/`, cada uno dentro de una transacción.

Migración	Aplicada en producción
`2026-09-27_tipos_vehiculo_y_adicionales.sql`	2026-09-27 12:26
`2026-10-08_refresh_tokens.sql`	pendiente

11.1 Despliegue con script (recomendado, desde el 2026-10-08)

```
cd /home/server-dt/Documentos/lavado_de_carros
./scripts/desplegar.sh --pull
```

Hace todo lo de abajo, en orden, y además **lleva anotado qué migraciones ya corrió** (tabla `_migracion_aplicada`): no se repiten ni se olvidan, que era el riesgo real del procedimiento manual. Termina esperando a `/api/health`; si la API no responde en 60 segundos, muestra el log del backend y falla.

Opción	Para qué
`--pull`	traer `origin/main` antes de desplegar
`--solo-migraciones`	aplicar migraciones sin tocar los contenedores

El respaldo sigue siendo manual y va primero (sección 13): el script no lo hace, y el ítem #17 del plan de trabajo —respaldos automáticos— sigue abierto.

Detalle completo en `docs/16_CI_CD/CI_CD.md`.

11.2 Procedimiento manual (el usado el 2026-09-27, corte de unos 30 s)

Sirve de referencia y para cuando haga falta hacer algo distinto:

```
cd /home/server-dt/Documentos/lavado_de_carros
# 1. Respaldo de base, archivos subidos y .env (ver sección 13)
# 2. Compilar las imágenes con el servicio aún en marcha
docker compose build backend frontend
# 3. Solo si hay migración: detener el backend y aplicarla
docker compose stop backend
docker exec -i lavado_de_carros-postgres-1 psql -U lavado_user -d lavado_db \
  -v ON_ERROR_STOP=1 < backend/prisma/migraciones/<archivo>.sql
# 4. Levantar y verificar
docker compose up -d backend frontend
curl -s http://127.0.0.1:3041/api/health
docker exec lavado_de_carros-backend-1 sh -c \
  'npx prisma migrate diff --from-url "$DATABASE_URL" --to-schema-datamodel
prisma/schema.prisma --exit-code'
# debe responder "No difference detected"
```

Para probar una migración antes de aplicarla: `pg_dump -Fc` de producción, `pg_restore` en un Postgres temporal y correr el SQL ahí.

Commits y push como `server-dt`, no como root (`sudo -u server-dt -H git …`): el repositorio tiene identidad local `megaman0012` y su remoto usa la clave personal.

12. App Android

Ver `15_APK/Preparacion_APK.md`. Estado al 2026-09-27: APK de **prueba** (`/home/server-dt/apk/TotalCleanCar-1.0-prueba.apk`), firmada con la clave de depuración, contra `http://181.188.232.50:3041` **sin cifrar**. Para publicarla en Google Play faltan un dominio con HTTPS y un keystore propio.

La APK trae su propia copia del frontend: **un cambio en la web no llega a la app hasta recompilarla** y reinstalarla.

13. Respaldos y recuperación

Desde el 2026-10-08 hay respaldo automático diario (cierra CONT-01 y el ítem

17 del plan). Detalle completo en

`12_CONTINUIDAD/Backup_Recuperacion.md`.

Script	`scripts/respaldo.sh`
Cron	diario a las **03:30**, como `server-dt`
Destino	`~/respaldos/lavado_de_carros/`
Retención	30 diarios + 12 mensuales (ver aviso)

> ⚠️ **Revisión 2026-10-10 (LAV-N05):** la retención mensual **todavía no funciona**: `respaldo.sh` lee mal el día del nombre del paquete y nunca reconoce el día 1. Hasta corregirlo solo existen los **30 diarios**. Además, los respaldos siguen en el mismo servidor y nada avisa si falta un día (el 2026-10-09 faltó por un corte del servidor).

Lo que hay que hacer de vez en cuando —toma cinco segundos:

```
./scripts/respaldo.sh --solo-verificar
```

Dice cuántos respaldos hay y **cuántos días tiene el más reciente**. Si pasan más de dos días, avisa: es la señal de que el cron dejó de correr.

⚠️ Lo que el respaldo todavía NO cubre

Está en el **mismo servidor** que el sistema (`DESTINO_EXTERNO` vacío). Cubre el borrado accidental, el error de migración y la corrupción de datos. **No cubre perder el servidor**: disco, incendio, robo. Para cerrarlo hay que definir `DESTINO_EXTERNO` apuntando a otro equipo, un NAS o un disco externo.

Qué contiene cada paquete

```
~/respaldos/lavado_de_carros/
  lavado_AAAA-MM-DD.tar.gz
    base/lavado_db.dump      pg_dump -Fc, verificado al generarse
    config/raiz.env          el .env con los secretos
    config/docker-compose.yml, nginx.conf
    repo/estado.txt          commit, rama y migraciones aplicadas
  uploads/                   espejo de evidencias y comprobantes (solo crece)
  log/respaldo.log           histórico de todas las corridas
```

Las fotos y comprobantes **no van dentro del .tar.gz**: cada lavado suma hasta 6 fotos y en un año serían gigas, multiplicadas por 30 días de retención. Van al espejo `uploads/`, que nunca borra: si alguien elimina evidencia del servidor, el espejo la conserva.

Antes de restaurar, **mirar** `repo/estado.txt`: dice qué commit estaba corriendo cuando se tomó el respaldo. Restaurar una base sobre un esquema distinto es adivinar.

También se conserva el respaldo manual previo al cambio a Total Clean Car, en `~/respaldos/lavado_de_carros/2026-09-27_pre-total-clean-car/`.

14. Mantenimiento

- **Base de datos:** PostgreSQL 15 con autovacuum por defecto. Sin tareas manuales para este volumen.
- **Archivos subidos:** el volumen `uploads_data` (884 KB al 2026-09-27) **crece sin límite**. No hay política de purga; conviene definir una retención antes de que escale.
- **Caché de la web:** el service worker pide `index.html` a la red primero y nginx lo sirve con `no-cache`, así que una versión nueva llega al recargar. Si se cambia `public/service-worker.js`, subir su `CACHE_NAME`.

15. Monitoreo y logs

```
docker compose ps
curl -s http://127.0.0.1:3042/api/health
docker compose logs -f backend
```

Sin monitoreo automático. El backend y el frontend no tienen healthcheck; solo PostgreSQL lo tiene.